

NAME: K. SWARNA VARSHINI

REG.NO: 192324081

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1715

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 3

Duration: 45 minutes

Total Marks: 30

Annexure A

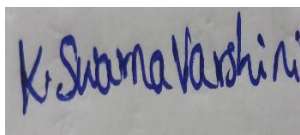
Constraint-Based Problem Solving

Code of Conduct:

I _K. Swarna Varshini_ (192324081) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift**
- ii. D2 must work before D3 (Morning < Afternoon < Night)**
- iii. Only one doctor per shift**
- iv. D3 cannot work Morning**
- v. All doctors must be assigned exactly one shift**

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right**
- ii. Cost of each move = 1**
- iii. Cannot pass through obstacles**

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a

survivor located at G.

Constraints:

- ☐ **The robot can move up, down, left, right**
- ☐ **Some cells are blocked (obstacles) and cannot be traversed**

- ☐ The robot has limited sensing capability (can only observe adjacent cells)
- ☐ Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- ☐ The robot must minimize total cost while ensuring safe navigation
- ☐ The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making

process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

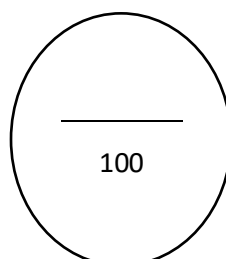
c) Demonstrate how the robot applies AI concepts such as:

- ☐ Intelligent agents
- ☐ Rationality
- ☐ Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the

given constraints.

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Total Marks (30)
Problem Understanding	25% (2.5)	/2.5	/2.5	/2.5	/7.5
Constraint Analysis	25% (2.5)	/2.5	/2.5	/2.5	/7.5
Solution Development	25% (2.5)	/2.5	/2.5	/2.5	/7.5
Application of Concepts	15% (1.5)	/1.5	/1.5	/1.5	/4.5
Justification	10% (1)	/1	/1	/1	/3
Total per Question	10 Marks	/10	/10	/10	/30



ANSWERS

1. Hospital Doctor–Shift Assignment Using Backtracking Search

Problem Statement

A hospital needs to assign three doctors **D1, D2, D3** to three shifts:

- Morning
- Afternoon
- Night

The assignment must satisfy all the given constraints.

Constraints

1. **D1 cannot work Night.**
2. **D2 must work before D3,** i.e.,
Morning < Afternoon < Night.
3. **Only one doctor per shift.**
4. **D3 cannot work Morning.**
5. **Every doctor must be assigned exactly one shift.**

Backtracking Search

Backtracking is a constraint satisfaction technique. It assigns values one by one and checks whether the current assignment satisfies all constraints. If a constraint is violated, the algorithm goes back and tries another assignment.

Step 1: Assign D1

Possible shifts for D1:

- Morning ✓
- Afternoon ✓
- Night ✗

Choose:

D1 → Morning

Current assignment:

Doctor	Shift
D1	Morning
D2	—
D3	—

Constraint checks:

- D1 is not working Night → ✓
- No duplicate shift → ✓

Step 2: Assign D2

Remaining shifts:

- Afternoon
- Night

Choose:

D2 → Afternoon

Current assignment:

Doctor	Shift
D1	Morning
D2	Afternoon
D3	—

Constraint checks:

- D1 is not Night → ✓
- D1 and D2 have different shifts → ✓
- D2 must be before D3 → D3 must be after Afternoon → ✓

Step 3: Assign D3

Remaining shift:

Night

Try:

D3 → Night

Current assignment:

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

Constraint Checks

Constraint	Check	Result
D1 cannot work Night	D1 = Morning	✓
D2 before D3	Afternoon < Night	✓
One doctor per shift	All shifts are different	✓
D3 cannot work Morning	D3 = Night	✓
All doctors assigned once	D1, D2, D3 assigned	✓

Backtracking Example

Suppose the algorithm tries:

D3 → Morning

This is invalid because:

D3 cannot work Morning.

Therefore:

Backtrack → Try D3 → Night

This satisfies all constraints.

Final Valid Schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Final Answer

D1 → Morning

D2 → Afternoon

D3 → Night

Hence, the assignment is valid and satisfies all constraints.

2. Robot Grid Navigation Using BFS Search

Problem Statement

A robot operates in a **5×5 grid** and must move from **Start (S)** to **Goal (G)**.

Constraints

- Movement allowed:
 - Up
 - Down
 - Left
 - Right
- Cost of every movement = **1**
- Obstacles cannot be crossed.

	1	2	3	4	5
1	S		#		
2			#		#
3					
4	#	#	#		
5				#	G

Where:

S = Start

G = Goal

= Obstacle

Blank cell = Free cell

BFS Principle

Breadth-First Search explores the grid level by level.

Since every movement has the same cost:

Cost = 1

BFS finds the path with the minimum number of movements.

BFS → FIFO Queue

Uniform Cost Search → Priority Queue

Because every movement costs 1, BFS gives the same optimal shortest path as Uniform Cost Search.

Therefore, below, the BFS FIFO queue is shown.

Step-by-Step BFS Node Expansion

Start:

$S = (1,1)$

Initial queue:

$[(1,1)]$

Step 1

Expand:

$(1,1)$

Possible movements:

Down → $(2,1)$ ✓

Right → $(1,2)$ ✓

Up → outside ✗

Left → outside ✗

Queue:

$[(2,1), (1,2)]$

Cost level = 1

Step 2

Expand:

$(2,1)$

New valid nodes:

Down → $(3,1)$ ✓

Right → $(2,2)$ ✓

Queue:

[(1,2), (3,1), (2,2)]

Step 3

Expand:

(1,2)

Neighbors:

Right → (1,3) = obstacle X

Down → (2,2), already visited

Up/Left → invalid

Queue:

[(3,1), (2,2)]

Step 4

Expand:

(3,1)

New node:

Right → (3,2) ✓

Queue:

[(2,2), (3,2)]

Step 5

Expand:

(2,2)

Its available neighbors are either obstacles or already visited.

Queue:

[(3,2)]

Step 6

Expand:

(3,2)

New node:

Right → (3,3) ✓

Queue:

[(3,3)]

Step 7

Expand:

(3,3)

New node:

Right $\rightarrow$ (3,4) ✓

Queue:

[(3,4)]

Step 8

Expand:

(3,4)

New valid nodes:

Up $\rightarrow$ (2,4) ✓

Down $\rightarrow$ (4,4) ✓

Right $\rightarrow$ (3,5) ✓

Queue:

[(2,4), (4,4), (3,5)]

Step 9

Expand:

(2,4)

New node:

Up $\rightarrow (1,4)$ ✓

Queue:

$[(4,4), (3,5), (1,4)]$

Step 10

Expand:

$(4,4)$

New node:

Right $\rightarrow (4,5)$ ✓

Queue:

$[(3,5), (1,4), (4,5)]$

Step 11

Expand:

$(3,5)$

No new valid nodes.

Queue:

$[(1,4), (4,5)]$

Step 12

Expand:

$(1,4)$

New node:

Right $\rightarrow$ (1,5) ✓

Queue:

[(4,5), (1,5)]

Step 13

Expand:

(4,5)

Move:

Down $\rightarrow$ (5,5) = Goal

Queue:

[(1,5), (5,5)]

The goal has been found.

BFS Expansion Table

Step	Expanded Node	New Node(s)	Queue
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1), (2,2)	(1,2), (3,1), (2,2)
3	(1,2)	None	(3,1), (2,2)
4	(3,1)	(3,2)	(2,2), (3,2)
5	(2,2)	None	(3,2)
6	(3,2)	(3,3)	(3,3)
7	(3,3)	(3,4)	(3,4)
8	(3,4)	(2,4), (4,4), (3,5)	(2,4), (4,4), (3,5)
9	(2,4)	(1,4)	(4,4), (3,5), (1,4)
10	(4,4)	(4,5)	(3,5), (1,4), (4,5)
11	(3,5)	None	(1,4), (4,5)
12	(1,4)	(1,5)	(4,5), (1,5)
13	(4,5)	(5,5) = G	Goal found

Optimal Path

The path obtained by tracing the parent nodes backward is:

(1,1)

↓

(2,1)

↓

(2,2)

↓

(3,2)

↓

(3,3)

↓

(3,4)

↓

(4,4)

↓

(4,5)

↓

(5,5)

Therefore:

S → (2,1) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4) → (4,5) → G

Cost Calculation

Number of movements = **8**

Cost of each movement = **1**

Therefore:

Total Cost = 8 × 1 = 8

Final Answer

Optimal Path Cost = 8

3. Autonomous Rescue Robot – Online Search

Problem Statement

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors.

The robot operates in a **partially observable environment**. It starts at **S** and must reach the survivor at **G**.

Given Constraints

- Movement allowed: Up, Down, Left, Right
- Obstacles cannot be traversed

- Robot can observe only adjacent cells
- Normal movement cost = 1
- Risky zone has an additional cost of 2
- Therefore, entering a risky zone costs:

$$1 + 2 = 3$$

- Robot must minimize total cost
 - Robot must dynamically update its knowledge
-

3(a). Analysis of Constraints

1. Movement Constraint

The robot can move only:

- Up
- Down
- Left
- Right

It cannot move diagonally.

Therefore, the search considers only four neighboring states.

2. Obstacles

Blocked cells cannot be entered.

When the robot detects an obstacle, that cell is removed from the possible actions.

3. Limited Sensing

The robot cannot see the complete building.

It can observe only adjacent cells.

Therefore, the robot does not initially know the entire environment and must make decisions with incomplete information.

4. Movement Cost

A normal movement costs:

1

The robot must consider the accumulated cost while selecting a path.

5. Risky Zones

A risky zone has:

- Normal movement cost = 1
- Additional risk cost = 2

Therefore:

Risky cell cost = 1 + 2 = 3

The robot should avoid risky zones when a safer lower-cost alternative exists.

6. Dynamic Knowledge

The robot discovers new information while moving.

For example:

Move → Sense → Update Map → Recalculate Path → Move

If a previously unknown path is found to be blocked, the robot must select another path.

3(b). Solution Approach – Uniform Cost Search + Online Search Agent

Selected Approach

The most suitable approach is:

Online Search Agent using Uniform Cost Search (UCS)

UCS selects the state having the **lowest accumulated path cost**.

This is suitable because the problem has **different movement costs**.

Cost Function

For a normal cell:

Cost = 1

For a risky cell:

Cost = 1 + 2 = 3

Steps of the Algorithm

Step 1

Start from **S** with cost:

$g(S) = 0$

Step 2

Sense adjacent cells.

Step 3

Identify whether each adjacent cell is:

- Free
- Obstacle
- Risky

- Goal

Step 4

Calculate the movement cost.

Normal cell:

Cost = 1

Risky cell:

Cost = 3

Step 5

Insert reachable cells into the UCS priority queue.

The queue is ordered according to accumulated cost.

Step 6

Select the node with the **lowest cost**.

Step 7

Move to the selected cell.

Step 8

Sense the surrounding cells again.

Step 9

Update the robot's internal map.

Step 10

If new information changes the available paths, perform the search again.

Step 11

Continue until the robot reaches **G**.

Example

Suppose the robot has two possible paths.

Path 1 – Short but Risky

$S \rightarrow A \rightarrow R \rightarrow G$

Cost:

$1 + 3 + 1 = 5$

Path 2 – Longer but Safe

$S \rightarrow B \rightarrow C \rightarrow D \rightarrow G$

Cost:

$1 + 1 + 1 + 1 = 4$

UCS compares the accumulated costs:

Path 1 = 5

Path 2 = 4

Therefore:

UCS selects Path 2.

3(c). AI Concepts Used

1. Intelligent Agent

An intelligent agent is a system that:

- Perceives its environment
- Processes information
- Takes actions
- Works toward a goal

The rescue robot is an **intelligent agent** because it:

Senses → Thinks → Acts → Learns/Updates

Sensors

The robot's sensors detect:

- Obstacles
- Risky areas
- Nearby rooms
- Goal/survivor

Actuators

The robot's actuators perform:

- Up movement
 - Down movement
 - Left movement
 - Right movement
 - Rescue action
-

2. Rationality

A rational agent chooses the best action based on the information currently available.

The rescue robot considers:

- Distance

- Movement cost
- Risk
- Obstacles
- Safety
- Newly discovered information

For example:

Path A = Cost 5

Path B = Cost 4

The rational decision is:

Choose Path B.

If Path B later becomes blocked, the robot updates its knowledge and selects another suitable route.

3. Environment Type

Property	Environment Type	Explanation
Observability	Partially Observable	Robot sees only adjacent cells
Agents	Single Agent	One rescue robot makes decisions
State	Discrete	Rooms/cells are separate states
Actions	Discrete	Up, Down, Left, Right
Nature	Sequential	Current actions affect future decisions
Knowledge	Initially Incomplete	Robot discovers information while moving
Cost	Non-uniform	Risky cells have higher cost

3(d). Why UCS + Online Search Is Suitable

1. Different Costs

BFS is best when every action has the same cost.

Here:

- Normal movement = 1
- Risky movement = 3

Therefore, **UCS is more suitable** because it considers actual accumulated cost.

2. Risk Awareness

Risky cells can be assigned higher costs.

This encourages the robot to choose safer paths when possible.

3. Partial Observability

The robot cannot see the complete environment.

An online search agent can:

Observe → Act → Learn → Update → Search Again

4. Dynamic Environment

If the robot discovers a new obstacle, it can update its map and find an alternative path.

5. Minimum-Cost Navigation

UCS is designed to find the path with the **minimum total cost** when action costs are non-uniform.

Therefore, the combination of:

Online Search Agent + Uniform Cost Search

is appropriate for this rescue robot.